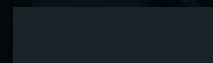
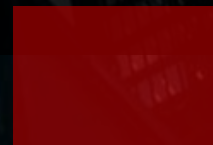
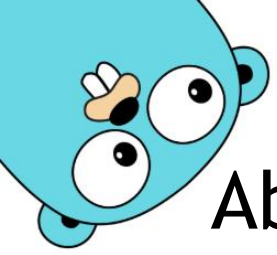


GoLang Basics



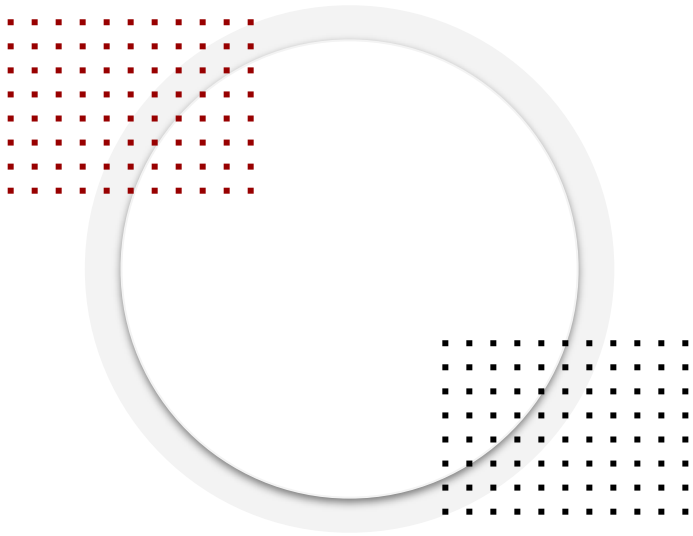
GO





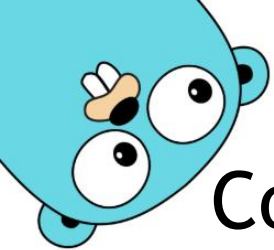
About Instructor

- About Instructor ... *<text size should be 16 and style should be Trebuchet MS>*



NAME

Datacouch Instructor



Course Roadmap

01

Language Fundamentals

Understand Go's unique design choices and syntax

03

Error Handling

Handle errors effectively; use panic/recovery safely

05

Concurrency

Use goroutines and channels for concurrent programming

02

Reusable Code

Write clean code with packages, modules, and functions

04

Structs & Interfaces

Work with methods, composition, and contract-based design

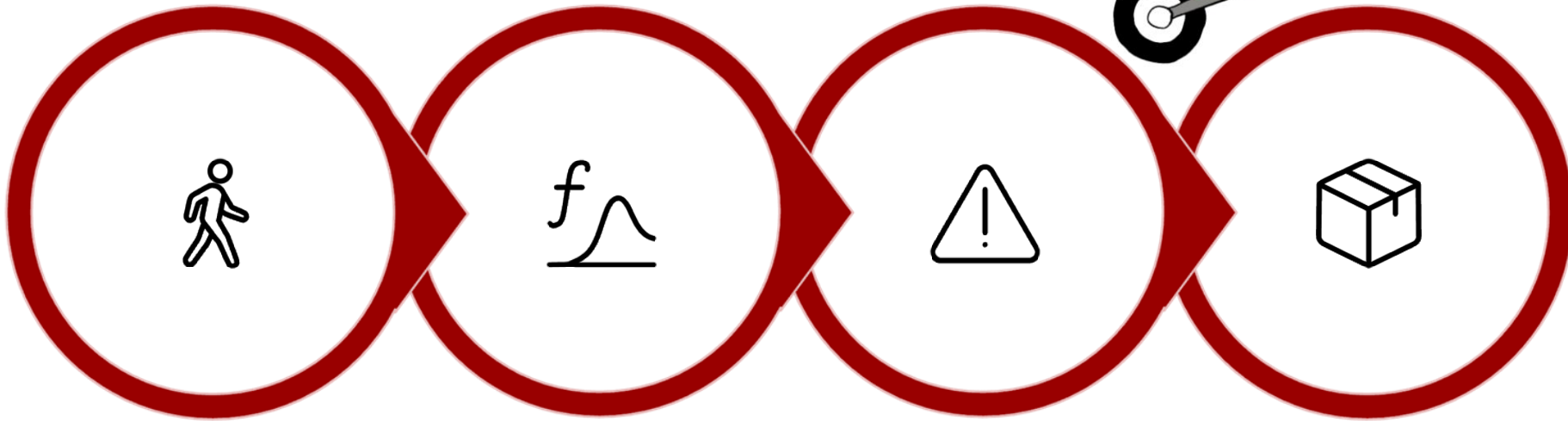
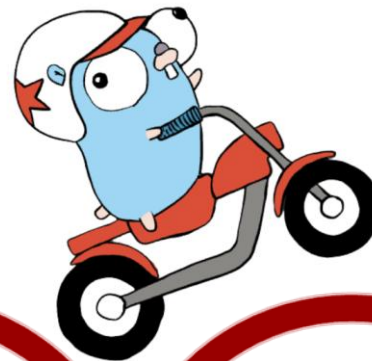
06

Web & Data

Build HTTP services, work with JSON and database/sql



What You Will Learn



Intro to Go

Functions & Types

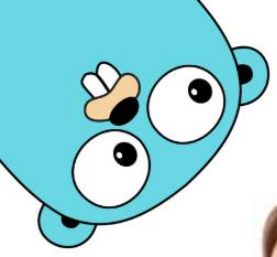
Error Handling

Packages & Tools

Each module builds on the previous one, taking you from first principles to production-grade Go code.

Introduction to Go

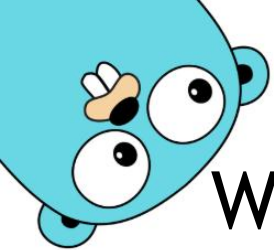




AGENDA

- Why Go? Benefits for enterprise software development
- Setting up the Go development environment
- Go program structure (packages, main, imports)
- Basic syntax: variables, constants, and data types
- Operators and expressions (high-level)
- Control flow statements (if/for/switch)



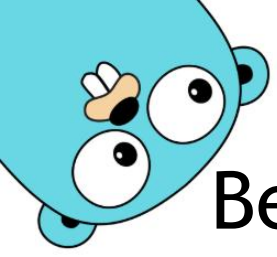


Why Go?

Go was created at Google in 2009 to solve real problems in large-scale systems engineering. It combines the performance of compiled languages with the simplicity of dynamically typed ones.

- Fast compilation and execution
- Built-in concurrency primitives
- Minimal, opinionated syntax
- Excellent standard library





Benefits for Enterprise Software



Performance

Compiles to native machine code — no runtime interpreter needed



Reliability

Static typing catches bugs at compile time, not in production



Scalability

Goroutines handle thousands of concurrent connections effortlessly



Simplicity

Small language spec means faster onboarding across large teams



Setting Up Your Environment

1 Install Go

Download from go.dev — supports macOS, Linux, and Windows

3 Pick an Editor

VS Code with the Go extension or GoLand are excellent choices

2 Verify Installation

Run `go version` in your terminal





Go Program Structure



Every Go programme follows a consistent structure built around three concepts:



Packages

Code is organised into packages. Every file declares its package at the top.



Imports

Dependencies are declared with **import**. Go enforces that unused imports are errors.



Main

Execution begins in **func main()** inside **package main.**



Your First Go Programme

Save this as `main.go`, then run `go run main.go`. That's it — no build step, no configuration files. Go's toolchain handles everything.



```
package main

import "fmt"

func main() {
    fmt.Println("Hello, Gophers!")
}
```



Variables and Constants

Variables

Declared with `var` or short assignment `:=`

```
var name string = "Go"  
age := 15 // type inferred
```

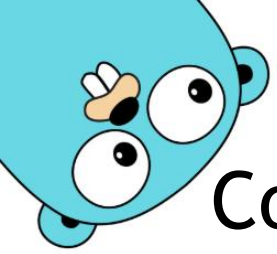
Go is **statically typed** — the compiler determines the type at compile time.

Constants

Declared with `const` — immutable values known at compile time.

```
const pi = 3.14159  
const lang = "Go"
```

Constants cannot use `:=` syntax.



Core Data Types

1

Numeric

`int`, `float64`, `uint`, `byte`



String

Immutable UTF-8
sequences



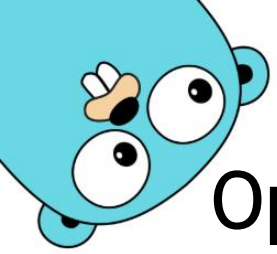
Composite

Arrays, slices, maps,
structs



Boolean

`true` or `false`



Operators and Expressions

Go supports the standard set of operators you'll recognise from other C-family languages, with a few notable differences.

Arithmetic

`+ - * / %`

Comparison

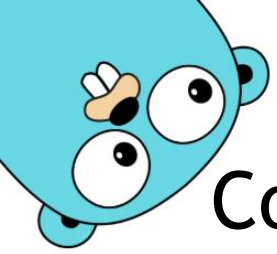
`== != < > <= >=`

Logical

`&& || !`



Go has no ternary operator. Use a standard `if/else` instead — the language favours clarity over brevity.



Control Flow: if / for / switch

if Statement

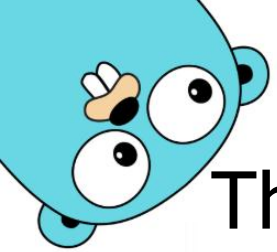
```
if x > 10 {  
    fmt.Println("big")  
} else {  
    fmt.Println("small")  
}
```

No parentheses around the condition — braces are always required.

for Loop

```
for i := 0; i < 5; i++ {  
    fmt.Println(i)  
}
```

Go has only one loop keyword: `for`. It replaces `while` and `do-while`.



The switch Statement

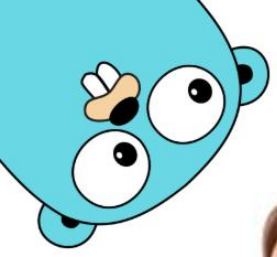


```
switch day {  
  case "Monday":  
    fmt.Println("Start of the week")  
  case "Friday":  
    fmt.Println("Almost weekend!")  
  default:  
    fmt.Println("Keep going")  
}
```

Go's **switch** doesn't need **break** - each case exits automatically. Use **fallthrough** only when you explicitly want to continue to the next case.

Functions, Types, and Structures





AGENDA

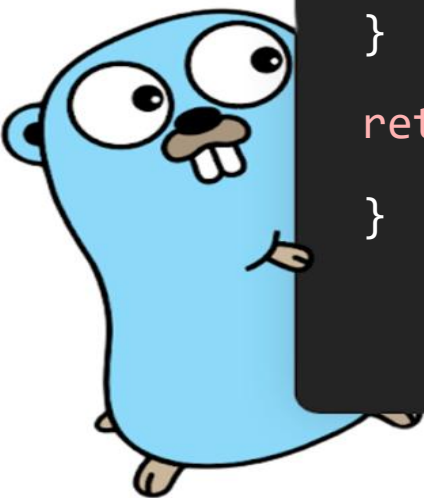
- Functions: parameters, return values, and multiple returns
- Anonymous functions and deferred execution (overview)
- User-defined types with struct
- Methods on structs and basic method patterns
- Interfaces: defining contracts (overview)
- Pointers (practical usage)





Functions in Go

Functions are first-class citizens in Go. They support multiple return values, which is fundamental to Go's error-handling philosophy.



```
func divide(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, errors.New("division by zero")  
    }  
    return a / b, nil  
}
```




Function Parameters & Returns

1

Parameters

Typed explicitly. Same-type params can be grouped: `func add(a, b int)`

2

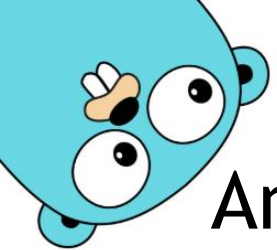
Multiple Returns

Idiom: return a result and an error —
`(value, error)`

3

Named Returns

Named return values act as documentation and can simplify short functions



Anonymous Functions & Defer

Anonymous Functions

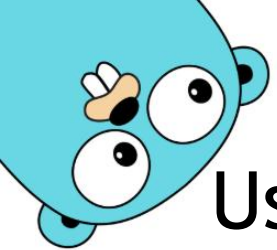
Functions without names — useful for inline callbacks and closures.

```
greet := func(name string) {  
    fmt.Println("Hello,", name)  
}  
greet("Gopher")
```

Defer

Schedules a function call to run **after** the surrounding function returns. Perfect for cleanup.

```
f, _ := os.Open("data.txt")  
defer f.Close()  
// file is closed automatically
```



User-Defined Types with Struct

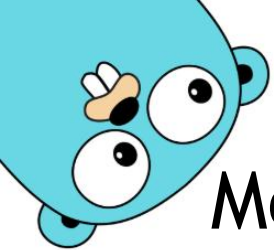
Structs are Go's primary way to group related data. They replace classes from other languages.

```
type User struct {  
    Name string  
    Email string  
    Age int  
}
```

```
u := User{Name: "Alice", Email: "alice@go.dev", Age: 30}  
fmt.Println(u.Name) // Alice
```



Go doesn't have classes or inheritance. Composition with structs and interfaces is the idiomatic approach.



Methods on Structs

Methods are functions attached to a type via a receiver. This gives structs behaviour.

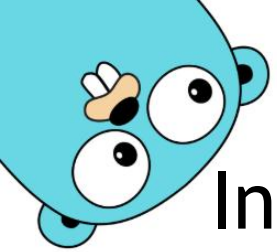
```
func (u User) Greet() string {  
    return "Hi, I'm " + u.Name  
}  
  
func (u *User) SetEmail(e string) {  
    u.Email = e  
}
```

Value vs Pointer Receivers

Value receiver — operates on a copy; original unchanged

Pointer receiver — can modify the original struct

Use pointer receivers when you need to mutate state or when the struct is large.

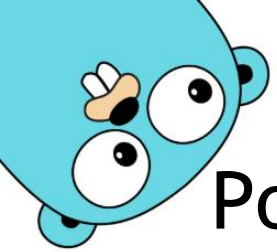


Interfaces: Defining Contracts

Interfaces define a set of method signatures. Any type that implements those methods **implicitly satisfies** the interface no explicit declaration needed.

```
type Greeter interface {  
    Greet() string  
}  
  
// User satisfies Greeter because  
// it has a Greet() string method
```

This is called **structural typing** and keeps your code flexible and decoupled.



Pointers: Practical Usage

What Are Pointers?

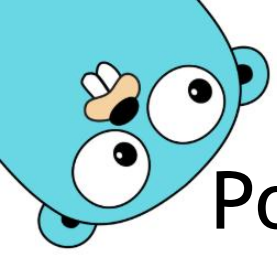
A pointer holds the **memory address** of a value. Declared with `*`, referenced with `&`.

Why Use Them?

Avoid copying large structs, allow functions to modify data in place, and enable shared state.

Safe by Default

Go has no pointer arithmetic. Dereferencing a nil pointer causes a clear panic — not undefined behaviour.

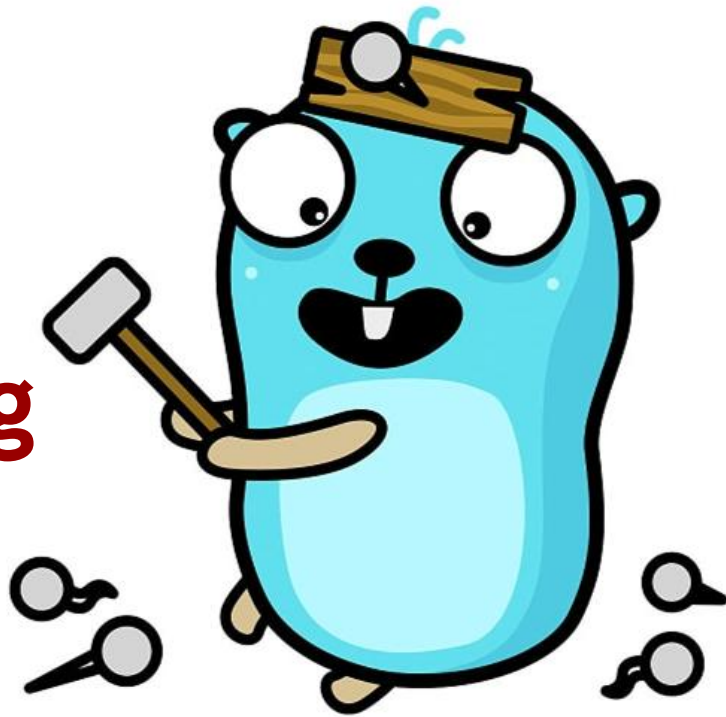


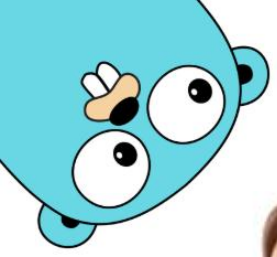
Pointers in Action

```
func double(val *int) {  
    *val = *val * 2  
}  
  
x := 5  
double(&x)  
fmt.Println(x) // 10
```

The & operator gets the address, * dereferences it. In practice, you'll use pointers most often with struct methods and function parameters.

Error Handling

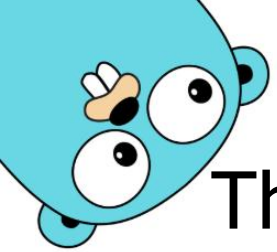




AGENDA

- The error interface and errors as values
- Creating and handling errors (`errors.New()`, `fmt.Errorf()`)
- Wrapping / nested errors (high-level)
- Panic and recover (used judiciously)
- Best practices for production-grade error handling



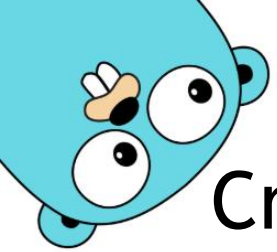


The Error Interface

In Go, errors are values, not exceptions. The built-in error interface is remarkably simple:

```
type error interface {  
    Error() string  
}
```

Any type with an Error() string method satisfies this interface. This makes errors composable, testable, and explicit.



Creating Errors

errors.New()

For simple, static error messages:

```
import "errors"

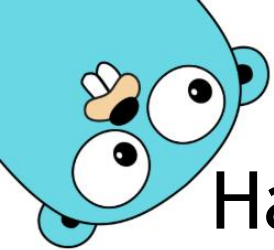
err := errors.New("user not found")
```

fmt.Errorf()

For dynamic errors with formatted context:

```
err := fmt.Errorf(
    "failed to load user %d: %w",
    id, originalErr,
)
```

The `%w` verb wraps the original error for inspection later.



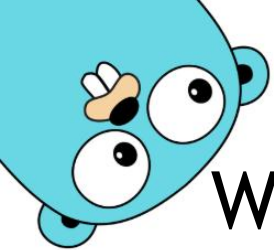
Handling Errors

The canonical pattern in Go checks errors immediately after every call that can fail.

```
result, err := doSomething()
if err != nil {
    return fmt.Errorf("doSomething failed: %w", err)
}

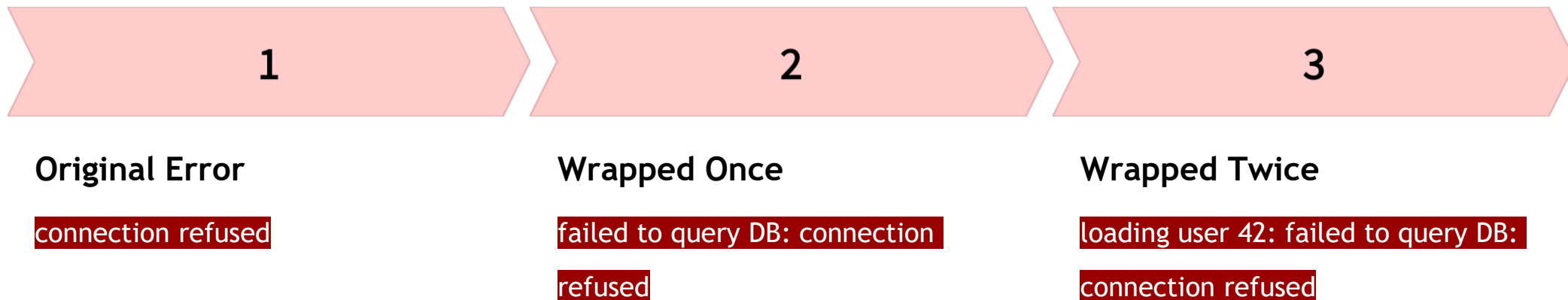
// continue with result
```

📌 Golden rule: Never ignore an error. If a function returns an error, handle it or propagate it with context.

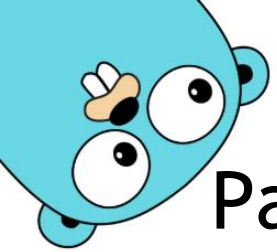


Wrapping & Nested Errors

Error wrapping creates a chain of context, making debugging in production far easier.



Use `errors.Is()` and `errors.As()` to inspect wrapped errors without string matching.



Panic and Recover

Panic

Stops normal execution and begins unwinding the stack. Use for truly unrecoverable situations.

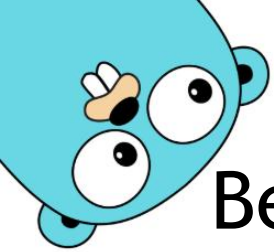
```
panic("critical: config missing")
```

Rule of thumb: if your programme can reasonably handle the situation, return an error instead.

Recover

Captures a panic inside a deferred function, allowing graceful shutdown.

```
defer func() {  
    if r := recover(); r != nil {  
        log.Println("recovered:", r)  
    }  
}()
```



Best Practices for Production Errors

Add Context

Wrap errors with

```
fmt.Errorf("doing  
X: %w", err)
```

 at every layer

Define Sentinel Errors

Use `var ErrNotFound = errors.New("not found")` for common conditions

Log at the Top

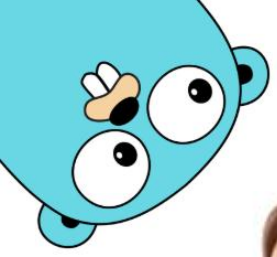
Return errors upward; log them once at the highest appropriate level

Avoid Panic in Libraries

Libraries should return errors. Only applications should decide to panic.

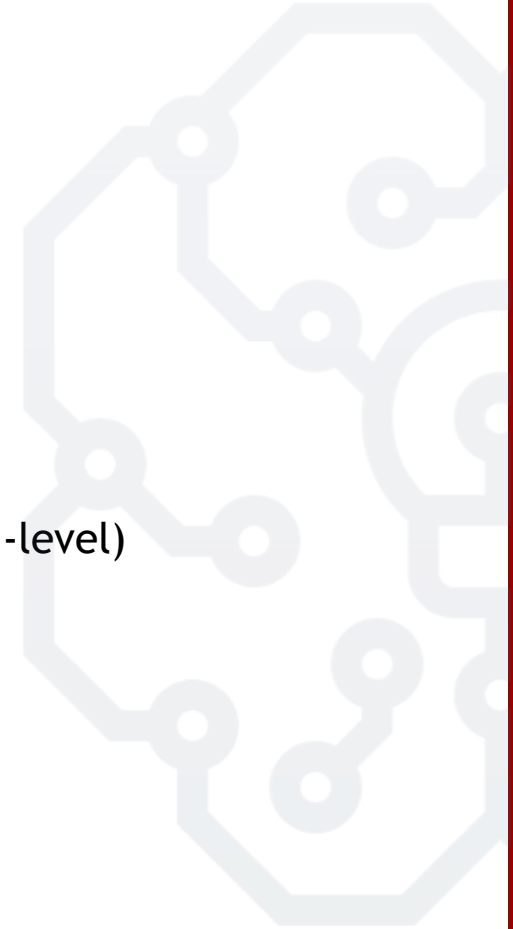


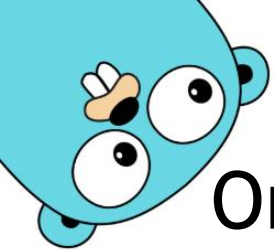
Packages, Modules, and Tooling



AGENDA

- Organizing code into packages
- Go Modules (go.mod) and dependency management
- Go tooling essentials: go fmt, go test, go mod, go vet (high-level)
- Testing with Go's testing package (unit test basics)





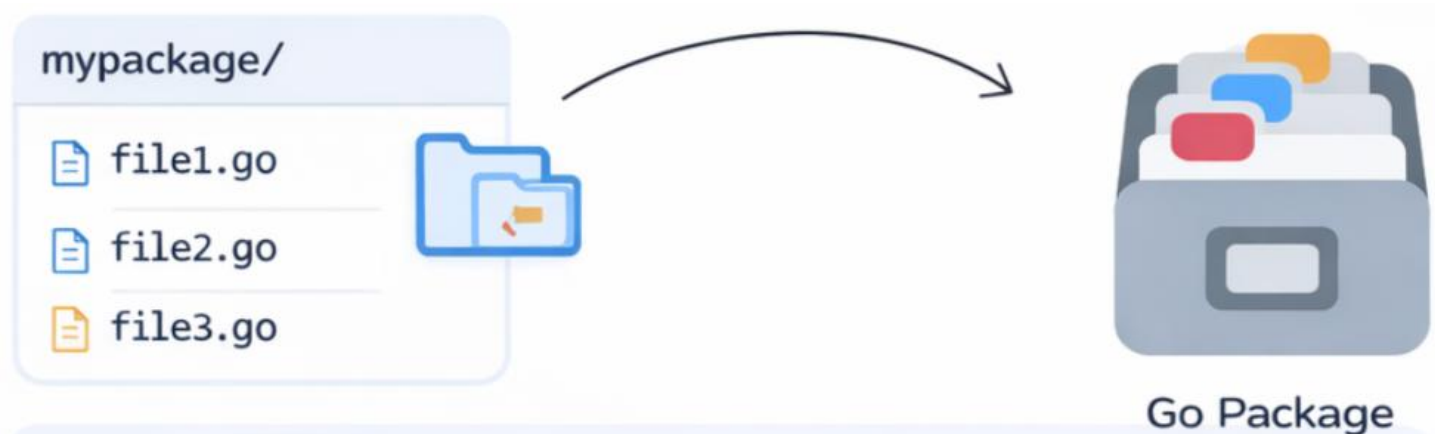
Organising Code into Packages

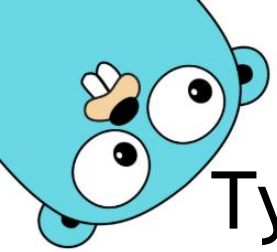
Every Go file belongs to a package. Packages provide **namespacing** and **encapsulation**.

- One package per directory

Exported names start with an uppercase letter

- Unexported names are package-private
- Keep packages small and focused on a single responsibility





Typical Project Structure

```
myapp/  
├─ go.mod  
├─ go.sum  
├─ main.go  
├─ handlers/  
│   └─ user.go  
├─ models/  
│   └─ user.go  
└─ services/  
    └─ auth.go
```

This layout separates concerns cleanly. The `go.mod` file at the root defines the module path and tracks dependencies.



Go Modules & go.mod

Go Modules are the official dependency management system, introduced in Go 1.11 and default since Go 1.16.

1 Initialise

Run `go mod init github.com/you/project`
to create a `go.mod` file

2 Add Dependencies

Use `go get` to fetch packages. Versions are
pinned in `go.sum`.

3 Tidy Up

Run `go mod tidy` to remove unused
dependencies and add missing ones



Go Tooling Essentials



go fmt

Formats your code to a canonical style — no more debates



go vet

Static analysis — catches suspicious constructs the compiler misses



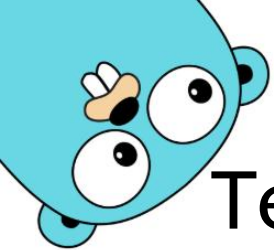
go test

Runs unit tests, benchmarks, and generates coverage reports



go mod

Module management — init, tidy, vendor, and download



Testing with Go

Go has a built-in testing framework — no third-party library required. Test files live alongside production code with a `_test.go` suffix.

```
func TestAdd(t *testing.T) {  
  
    // Call the Add function with sample inputs  
    got := Add(2, 3)  
  
    // Verify the result matches the expected value  
    if got != 5 {  
  
        // Fail the test immediately if the result is incorrect  
        t.Fatalf("Add(2,3) = %d; want 5", got)  
    }  
}
```



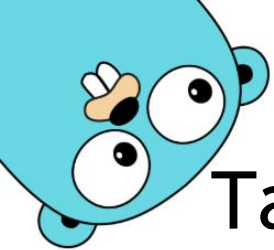
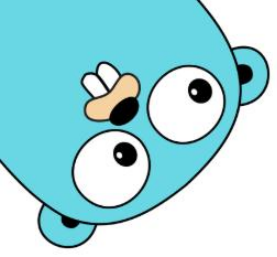


Table-Driven Tests

The idiomatic Go approach for testing multiple cases concisely. Define inputs and expected outputs in a slice, then loop through.

```
tests := []struct {  
    a, b, want int // inputs (a, b) and expected output (want)  
}{  
    {1, 2, 3}, // normal case  
    {0, 0, 0}, // edge case: zero values  
    {-1, 1, 0}, // edge case: negative + positive  
}  
  
// Loop through each test case  
for _, tt := range tests {  
  
    // Execute the function with current inputs  
    got := Add(tt.a, tt.b)  
  
    // Validate the result  
    if got != tt.want {  
  
        // Report mismatch between actual and expected result  
        t.Errorf("Add(%d,%d) = %d; want %d", tt.a, tt.b, got, tt.want)  
    }  
}
```



A low-angle, upward-looking photograph of several modern skyscrapers with glass facades. The image is overlaid with a semi-transparent dark grey horizontal band across the middle. Above this band, a large red rectangle is positioned on the left, and a smaller red rectangle is on the right. Below the band, a red rectangle is on the left, and a larger red rectangle is on the right. The text "THANK YOU" is centered in the dark grey band in a white, bold, sans-serif font.

THANK YOU